



PROGRAMACION ESTRUCTURADA BASICA

UNIDAD 5. Introducción a Punteros

INDICE

1.	¿QUÉ SON LOS PUNTEROS?	2
2.	DECLARACIÓN DE PUNTEROS	2
3.	ASIGNACIÓN DE PUNTEROS.....	3
4.	ACCEDER A LO APUNTADO POR UN PUNTERO.....	3
5.	ARITMÉTICA DE PUNTEROS	4
6.	PASAJE POR REFERENCIA	4
7.	MANEJO DE VECTORES CON PUNTEROS	5
8.	PUNTEROS A ESTRUCTURAS	8
9.	PUNTERO A NULL.....	9
10.	AREAS DE MEMORIA DE UN PROGRAMA EN C	9
11.	ASIGNACIÓN DINÁMICA DE MEMORIA.....	10
	11.1. MALLOC()	11
	11.2. CALLOC()	12
	11.3. REALLOC()	13
	11.4. FREE()	13
12.	ERRORES COMUNES EN EL USO DE MEMORIA DINÁMICA	14
	12.1. MEMORY LEAK (FUGA DE MEMORIA)	14
	12.2. DANGLING POINTER (PUNTERO COLGANTE)	14
	12.3. DOUBLE FREE	15
	12.4. PERDER EL PUNTERO (LOST POINTER)	15
	12.5. ACCESO FUERA DE LÍMITES (BUFFER OVERFLOW)	15

UNIDAD 5

Introducción a Punteros

OBJETIVOS: Tener un primer acercamiento al uso de punteros comprendiendo el concepto y aplicaciones básicas de utilización

1. ¿Qué son los punteros?

Un puntero es una variable que en su interior guarda una dirección de memoria. Se lo denomina puntero porque al tener la dirección de otra variable es una referencia a otro lugar, “apunta” hacia otra dirección de memoria.

La dirección de memoria que almacena puede ser de otra variable de cualquier tipo ya sea un tipo básico, un tipo creado por el usuario como una estructura, o incluso puede tener la dirección de otra variable del tipo puntero. También existen los llamados punteros a funciones que no son tema de esta materia.

Algunos de los usos de este tipo de variables son:

1. Leer o escribir cualquier posición de memoria (Ej. Memoria de Video)
2. Transferencia a funciones por “Referencia”
3. Soportar las rutinas de asignación dinámica de memoria permitiendo el uso de estructuras avanzadas de datos como:
 - Listas enlazadas
 - Listas doblemente enlazadas
 - Pilas
 - Colas
 - Arboles
4. Operar más eficientemente en los arrays que mediante subíndices

2. Declaración de punteros

A la variable del tipo puntero si bien guarda una dirección de memoria se le debe indicar a que tipo de dato corresponde la variable apuntada, lo que se denomina tipo de dato base.

Un puntero se declara anteponiendo un asterisco al identificador de la variable al declararla del siguiente modo:

```
tipo de dato *identificador;
```

Por ejemplo:

```
int *p; //define una variable llamada p que apunta a un tipo de dato entero
```

```
float *pf; //define una variable llamada pf que apunta a un tipo de dato float  
char *pc; //define una variable llamada pc que apunta a un tipo de dato char  
int num, * pi; //declara una variable entera y un puntero a entero  
int *p1, *p2; //declara dos punteros a entero
```

Si bien todas las variables del tipo puntero ocupan lo mismo en memoria (el tamaño de la dirección) el tipo base es importante y será utilizado por el compilador al momento de utilizar aritmética de punteros que se explica en el punto 5.

Al momento de declarar una variable del tipo puntero como toda variable del lenguaje C inicialmente tiene un valor al azar del contenido previo de la memoria lo que comúnmente se denomina “basura de memoria” por lo tanto al puntero se lo debe “controlar” es decir asignarle una dirección válida. Se dice que un puntero no está controlado cuando apunta a un lugar al azar.

3. Asignación de punteros

Un puntero puede ser asignado de 4 formas:

1. Con la dirección de otra variable

```
int a, *p;  
  
p = &a;
```

2. A través de otro puntero

```
int a, *p, *q;  
  
p = &a;  
  
q = p;
```

3. Directamente con la dirección que deba contener

```
int *p = (int *) 0xBCFEAA02;
```

4. Mediante funciones para el manejo de memoria dinámica (malloc y realloc)

4. Acceder a lo apuntado por un puntero

Como el puntero guarda una dirección de memoria, si se quiere mediante dicho puntero llegar al dato apuntado se debe utilizar el operador asterisco.

Por ejemplo, el siguiente código muestra el contenido de la variable a mediante el puntero:

```
int a=10, *p;  
  
p = &a;  
  
printf("Valor de a: %d", *p);
```

Por lo tanto, el operador asterisco antepuesto a una variable del tipo puntero lo que indica es “El contenido de lo apuntado por dicha variable”. Sirve tanto para visualizar el dato apuntado como para modificarlo, en el mismo ejemplo la siguiente instrucción cambiaría el valor de la variable a por un 20:

```
*p = 20;
```

5. Aritmética de Punteros

A un puntero se le puede sumar o restar un entero. Esta suma va a modificar la dirección de memoria a la cual apunta.

La unidad de incremento o decremento es la cantidad de bytes que ocupa el tipo de dato al cual apunta. Por ejemplo, si se tiene un puntero a entero al sumar 1 se le suman 4 bytes a la dirección guardada en el puntero ya que es el tamaño de una variable entera. En cambio, si el puntero es a char, a sumarle uno solo se le sumará un byte a la dirección.

También es posible restar dos punteros del mismo tipo y asignar el resultado de la resta a una variable entera. Dicha variable indica la distancia entre las dos direcciones representadas en grupos de bytes según el tipo de dato al cual apuntan. No es posible sumar, multiplicar ni dividir punteros, solo restar.

6. Pasaje por referencia

Si a una función se le envía un puntero, se le está enviando la dirección de memoria de una variable definida en otro bloque. Dentro de la función al acceder a dicha variable se estará accediendo a la memoria fuera del ámbito de la función por lo tanto es una referencia al dato.

Veamos el siguiente ejemplo: Se ingresa precio unitario y cantidad de unidades vendidas. Se debe calcular el importe final aplicando un descuento del 30% para ventas de más de 100 unidades, del 20% para ventas de más de 50 unidades o del 10% para ventas de más de 30 unidades. Se debe realizar una función que calcule el importe a abonar y retorne el porcentaje de descuento aplicado.

Para resolver este problema como la función no puede retornar dos valores (el descuento y el importe) se enviará la variable donde quedará almacenado el importe final por referencia, a fin de que el valor modificado en la función se vea reflejado en el programa principal. Para ello se recibe la dirección de memoria de dicha variable en un puntero.

```
#include <stdio.h>
int CalcularImporteFinal(float*, int);
int main()
{
    float importe;
    int cantidad, porcDesc;
    printf("Ingrese el precio unitario:");
    scanf("%f", &importe);
    printf("Ingrese la cantidad vendida:");
    scanf("%d", &cantidad);
    porcDesc = CalcularImporteFinal(&importe, cantidad);
    if (porcDesc > 0)
        printf("\nSe aplico un descuento del %d%%", porcDesc);
    printf("\nTotal a pagar: %.2f", importe);
    return 0;
}

int CalcularImporteFinal(float *imp, int cant)
{
    int desc = 0;
    if (cant > 100)
        desc = 30;
    else
        if (cant > 50)
            desc = 20;
        else
            if (cant > 30)
                desc = 10;
    *imp *= cant;
    *imp -= ((*imp) * desc) / 100;
    return desc;
}
```

La variable imp trabaja referenciando a la variable del main por lo tanto los datos se modifican en la variable importe del main al trabajar con su dirección de memoria.

7. Manejo de vectores con punteros

Al momento de definir un vector lo que queda almacenado en la variable de este tipo es la dirección de memoria donde se encuentra el primer dato. Es decir, un vector es un puntero, pero es un puntero del tipo estático ya que esa dirección de memoria no puede ser modificada.

Al trabajar con subíndices lo que hace el compilador es calcular las direcciones de memoria a la cual debe acceder. Pero existe otra forma de acceder a las posiciones del vector trabajando directamente con la dirección de memoria de inicio y aplicando aritmética de punteros.

Siendo vec un vector, las equivalencias son las siguientes:

`vec+i` es equivalente a `&vec[i]`

`*(vec+i)` es equivalente a `vec[i]`

Por lo tanto, es posible recorrer un vector utilizando dichas equivalencias, veamos un ejemplo de una función que carga datos en un vector y otra que lo muestra.

```
#include <stdio.h>
void carga (int[], int);
void mostrar(int[], int);

int main()
{
    int vec[5];
    carga (vec,5);
    mostrar(vec,5);
    return 0;
}
```

Hasta este punto no hay diferencia con el método tradicional, es más la declaración de prototipos y llamadas a funciones es igual que si se utiliza subíndices, la diferencia está en el desarrollo de las funciones:

```
void carga (int v[], int tam)
{
    int i;
    for (i=0;i<tam;i++)
    {
        printf ("ingrese un numero: ");
        scanf("%d", v+i);
    }
}
```

En la función carga se está utilizando aritmética de punteros para ir cambiando la dirección en la cual se va a realizar la lectura.

```
void mostrar (int v[], int tam)
{
    int i;
    for (i=0;i<tam;i++)
        printf("%d\n", *(v+i));
}
```

En la función mostrar se accede primero a la dirección que se quiere visualizar sumándole i y luego se accede al contenido dicha dirección con el operador asterisco.

Otra forma de declarar los vectores tanto en los prototipos de funciones como en la especificación de parámetros sería indicando que se recibe una dirección mediante un puntero. Pero esto no afecta en nada ni cambia como se maneja internamente el acceso en la función. A continuación, se muestra la función mostrar con dicha nomenclatura en la declaración de parámetros:

```
void mostrar (int *v, int tam)
{
    int i;
    for (i=0;i<tam;i++)
        printf ("%d\n", *(v+i));
}
```

Existe otra forma de recorrer los vectores utilizando nomenclatura de punteros, pero cambiando directamente la dirección a la que apunta el puntero en lugar de sumarle la constante. A continuación, se muestra el mismo ejemplo pero con esta forma de trabajo tomando en cuenta de que el main es el mismo.

```
void carga (int v[], int tam)
{
    int i;
    for (i=0;i<tam;i++)
    {
        printf ("ingrese un numero: ");
        scanf ("%d", v);
        v++;
    }
}

void mostrar (int v[], int tam)
{
    int i;
    for (i=0;i<tam;i++)
    {
        printf ("%d\n", *v);
        v++;
    }
}
```

También puede resolverse comparando y modificando las direcciones en el propio for:

```
void carga (int v[], int tam)
{
    int *p;
    for (p=v;p<v+tam;p++)
    {
        printf ("Ingrese un numero: ");
        scanf ("%d", p);
    }
}

void mostrar (int v[], int tam)
{
    int *p;
    for (p=v;p<v+tam;p++)
        printf ("%d\n", *p);
}
```

8. Punteros a estructuras

Cuando el puntero es a una estructura, primero se debe acceder a la variable mediante el puntero y luego al campo por ejemplo dada la siguiente estructura:

```
#include <stdio.h>
typedef struct
{
    char nombre[21];
    int dni;
}sAlumno;
```

En el main se inicializa una variable con valores y mediante un puntero a estructura se los muestra:

```
int main()
{
    sAlumno alumno={"Analía", 42772521};
    sAlumno *pAl;
    pAl = &alumno;

    printf ("Nombre: %s", (*pAl).nombre);
    printf ("\nDNI: %d", (*pAl).dni);
    return 0;
}
```

Nótese que hay que utilizar paréntesis para llegar al contenido de la dirección antes de poder utilizar el operador asterisco.

Para facilitar la nomenclatura que puede llegar a ser engorrosa en estos casos el lenguaje prevee la utilización del operador flecha (->) formado por los símbolos menos y mayor. Este operador puede utilizarse de forma directa para acceder al contenido de un campo apuntado por un puntero a estructura de la siguiente forma:

```
int main()
{
    sAlumno alumno={"Analía",42772521};
    sAlumno *pAl;
    pAl = &alumno;

    printf("Nombre: %s", pAl->nombre);
    printf("\nDNI: %d", pAl->dni);
    return 0;
}
```

9. Puntero a NULL

En ciertas ocasiones hay que desarrollar funciones que deben retornar una dirección de memoria, por ejemplo, la dirección donde se encuentra determinado dato en un vector. ¿Pero que pasa si el dato no se encuentra? ¿Como se expresa que no se encontró mediante una dirección?

En esos casos se utiliza la constante NULL asignarle la constante NULL a una variable tipo puntero se utiliza para indicar que la dirección que guarda no es válida. Entonces pueden hacerse comparaciones para ver si la dirección guardada es válida o no.

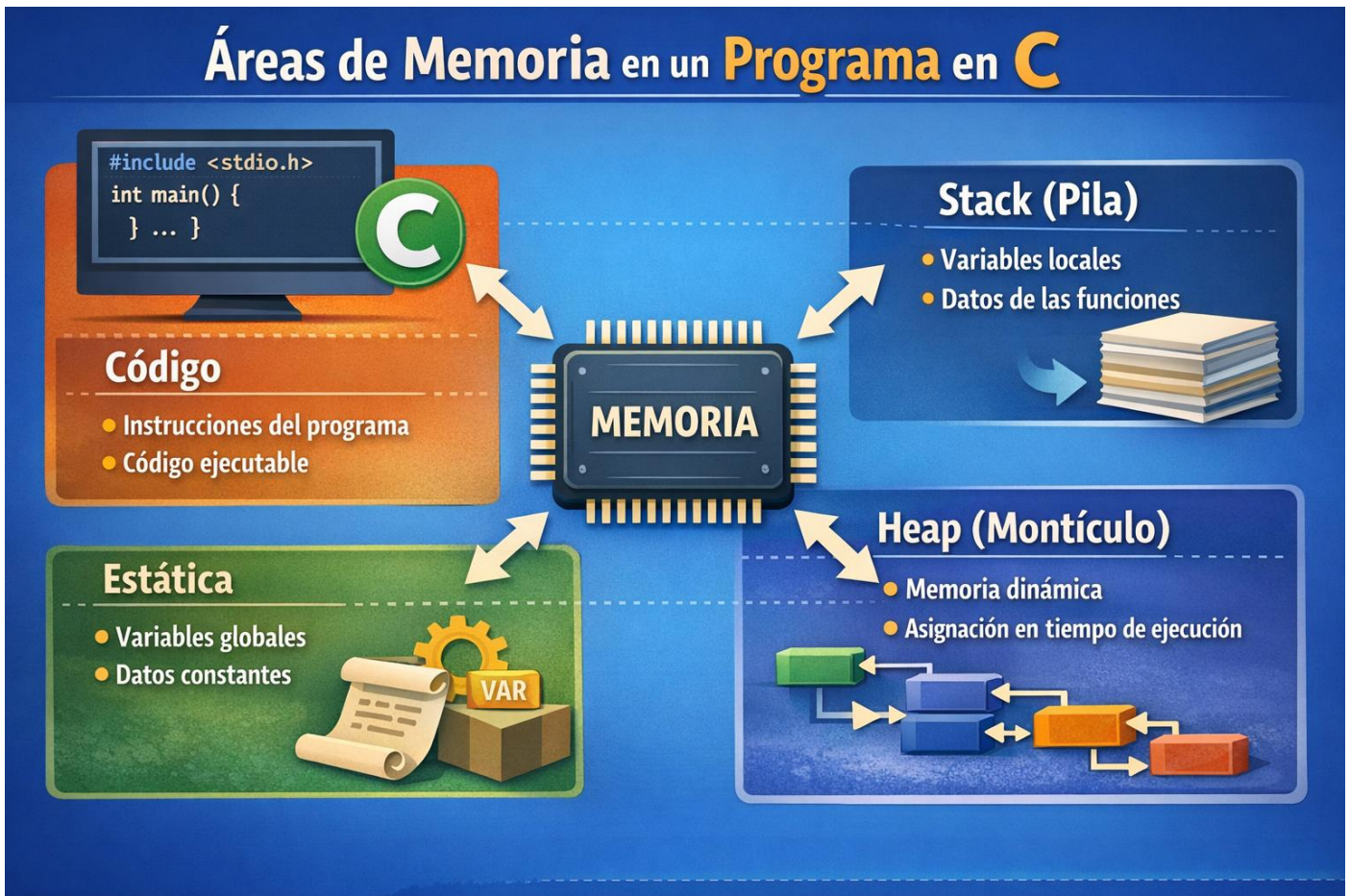
Siendo p un puntero una comparación válida sería `if (p!=NULL)`

En general en las funciones que retornan punteros se asigna el valor NULL en el caso de que falle el procedimiento de la función o no se encuentre el valor esperado.

10. Areas de memoria de un programa en C

Cuando un programa está en ejecución la memoria se divide en distintos sectores:

- Sector de código, donde se almacenan las instrucciones del programa que se está ejecutando
- Area estática, donde se almacenan las variables globales, variables estáticas o constantes (variables definidas con modificador `const`)
- Pila o Stack: donde se almacenan las variables locales y variables creadas por parámetros formales de funciones. Además la pila guarda las direcciones de retorno a la cual es programa debe volver una vez que sale de una función.
- Heap: área de memoria disponible para ser solicitada/liberada en tiempo de ejecución (memoria dinámica)



11. Asignación dinámica de memoria

Hasta ahora siempre al diseñar nuestros programas teníamos que dimensionar nuestras colecciones de datos con un máximo establecido según el problema a resolver, esto hace que se desperdicie memoria ya que se deben sobredimensionar los vectores y matrices para asegurarnos de que el peor caso los datos puedan guardarse. El uso de punteros nos abre la puerta a trabajar con otra área de memoria que puede ser solicitada y liberada en tiempo de ejecución del programa, a esta memoria se la conoce como memoria dinámica.

Además, el área de la pila tiene un tamaño máximo, que no siempre es el mismo sino que está definido por el sistema operativo. Por lo tanto si se necesita trabajar con muchos datos podría desbordarse es pila, es decir que se llega a su capacidad máxima y el programa intenta escribir por fuera del espacio reservado, lo que comúnmente se conoce como stack overflow.

Para trabajar con memoria dinámica se utilizan 4 funciones

Función	Uso	Descripción
malloc()	Reservar memoria	Reserva un bloque de memoria de tamaño especificado
calloc()	Reservar e inicializar	Reserva memoria para varios elementos e inicializa en 0
realloc()	Redimensionar memoria	Cambia el tamaño de un bloque previamente reservado
free()	Liberar memoria	Libera memoria reservada dinámicamente

El detalle del uso de las 4 funciones que se explican a continuación esta basado en libro “C para ingeniería electronica”, autor Jorge A. Argibay y Mauro Gullino.

11.1. **MALLOC()**

Mediante la función malloc() se solicita la reserva de un bloque de memoria de un determinado tamaño. Su prototipo es:

```
void *malloc(size_t tamaño);
```

La función malloc() requiere que se le informe el tamaño en bytes del bloque libre que se desea obtener.

El tipo definido como size_t se utiliza en la biblioteca estándar para representar cantidades enteras positivas, como ser un tamaño en bytes o una determinada cantidad de bloques.

El resultado entregado por malloc() es la dirección inicial de un bloque de memoria libre si la gestión tuvo éxito, o el puntero NULL si no se asignó memoria.

Dado que malloc() no sabe para que se está gestionando el bloque de memoria, retorna la dirección de este con formato void*. Es necesario castear este tipo a fin de asignarlo al puntero correspondiente.

Es necesario siempre comprobar que la dirección retornada por malloc() sea una dirección valida. Esto significa que sea diferente de NULL.

Es importante asegurar la portabilidad de los tamaños gestionados dinámicamente en la memoria a fin de no exceder los límites de la memoria asignada. Esta portabilidad se logra utilizando el operador sizeof() para establecer los tamaños.

A continuación, se muestra la gestión de memoria para una estructura, considerando todos los puntos anteriores.

Nótese que se evalúa la condición de falso del valor asignado al puntero pData. Esto es debido a que la evaluación lógica de NULL es falso, puesto que su valor numérico es cero.

En caso de detectarse tal situación, se informa y se termina el proceso.

```
#include<stdio.h>
#include<stdlib.h>

typedef struct
{
    char nombre[20];
    char sexo;
    int edad;
}Dato;

int main()
{
    Dato *pDato;

    if(! (pDato = (Dato*) malloc(sizeof(Dato))))
    {
        printf("No hay suficiente memoria");
        exit(1);
    }

    return 0;
}
```

11.2. **CALLOC()**

La funcion calloc() es similar a la funcion malloc() en cuanto a la gestion de memoria. La diferencia esta en que calloc() permite expresar el tamaño de memoria solicitado, mediante una cierta cantidad de bloques de una determinada magnitud. Además a diferencia de malloc esta función inicializa automáticamente todos los bits del bloque de memoria asignado en cero.

El prototipo es:

```
void calloc(size_t numero, size_t tamaño)
```

El tamaño de la memoria asignada sera igual al numero multiplicado por el tamaño por lo que se podria haber utilizado en su lugar malloc(numero*tamaño) con la unica diferencia de que la memoria no estaria inicializada.

A continuacion, se muestra la gestion de memoria para un vector de 10 flotantes utilizando calloc().

```
#include<stdio.h>
#include<stdlib.h>

#define N 10

int main()
{
    float *pFloat;

    if(! (pFloat = (float*) calloc(N, sizeof(float))))
    {
        printf("No hay suficiente memoria");
        exit(1);
    }

    return 0;
}
```

11.3. REALLOC()

Esta funcion permite reasignar memoria asignado a un bloque. Puede ocurrir que el tamaño de un bloque solicitado anteriormente ya no sea el adecuado, por ser demasiado chico o por ser demasiado grande. Su prototipo es:

```
void *realloc(void *ptr, size_t tamaño)
```

La funcion realloc() requiere que se le entregue la direccion del bloque a reasignar y el nuevo tamaño deseado.

Puede ocurrir que realloc() no pueda redimensionar el bloque en el lugar asignado anteriormente. De ser asi, gestiona un nuevo bloque sobre el cual copia la informacion del anterior y retorna su direccion.

En caso de no poder reasignar dicho bloque, realloc() retorna el puntero NULL.

```
#include<stdio.h>
#include<stdlib.h>

#define N 10

int main()
{
    //Se reserva un bloque de memoria
    int *pInt = (int*) malloc(N);

    //Se reasigna el tamaño del bloque anterior
    pInt = (int*) realloc(pInt, 3 * N);

    return 0;
}
```

11.4. FREE()

La funcion free() libera un bloque de memoria gestionado anteriormente a fin de poder asignarlo nuevamente. Su prototipo es:

```
void free(void *ptr);
```

La función `free()` requiere que le entregue la dirección de un bloque de memoria gestionado por alguna de las funciones anteriormente descritas.

El argumento entregado a la función `free()` debe provenir de una asignación dinámica anterior.

Es muy importante siempre liberar la memoria solicitada en forma dinámica, principalmente en las funciones ya que si al entrar a una función se solicita un bloque de memoria dinámica que luego no se libera cuando se sale de la función esa memoria queda inutilizable e inalcanzable por el programa. Es memoria perdida que solo se puede recuperar por el sistema operativo al finalizar el programa. A este problema se lo conoce como memory leak. Pero ese no es el único problema que puede ocurrir.

12. Errores comunes en el uso de memoria dinámica

En C, trabajar con memoria dinámica puede generar varios errores típicos. Estos son los 5 errores más comunes

- Memory Leak (fuga de memoria)
- Dangling Pointer (puntero colgante)
- Double Free (liberar dos veces)
- Perder el puntero (Lost Pointer)
- Acceso fuera de límites (Buffer Overflow)

12.1. **Memory Leak (fuga de memoria)**

Ocurre cuando se reserva memoria pero nunca se libera.

Ejemplo:

```
int *p = malloc(sizeof(int));  
  
*p = 5;  
  
// falta free(p)
```

La memoria queda ocupada hasta que termina el programa, que trae por consecuencia consumo creciente de RAM, y programas que se vuelven lentos

12.2. **Dangling Pointer (puntero colgante)**

Sucede cuando se libera la memoria pero el puntero sigue apuntando a ella.

Ejemplo:

```
int *p = malloc(sizeof(int));  
  
free(p);  
  
*p = 10;    // ERROR
```

Se accede a memoria **ya liberada** lo que el comportamiento será indefinido y es probable que el programa de un error en tiempo de ejecución

Solución recomendada:

```
free(p);  
p = NULL;
```

12.3. **Double Free**

Ocurre cuando se libera la misma memoria dos veces.

Ejemplo

```
int *p = malloc(sizeof(int));  
  
free(p);  
free(p);    // ERROR
```

El sistema intenta liberar memoria que ya fue liberada lo que llevará a error en ejecución.

12.4. **Perder el puntero (Lost Pointer)**

Sucede cuando se pierde la referencia al bloque de memoria.

Ejemplo

```
int *p = malloc(sizeof(int));  
  
p = malloc(sizeof(int));    // se pierde el primer bloque
```

La memoria anterior ya no puede liberarse lo que produce un Memory Leak

12.5. **Acceso fuera de límites (Buffer Overflow)**

Ocurre cuando se accede fuera del espacio reservado.

Ejemplo

```
int *v = malloc(3 * sizeof(int));  
  
v[0] = 1;  
v[1] = 2;  
v[2] = 3;  
v[3] = 4;    // ERROR
```

Se escribe fuera de la memoria reservada lo que puede sobrescribir otros datos o dar errores al acceder a memoria no reservada.

Este es un error común también que se da al manejar arrays estáticos en la pila